

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - AĞ TABANLI PARALEL DAĞITIM
SİSTEMLERİ**

PROJE - 5

E-Ticaret Verilerinde ETL Süreci: Northwind Örneği

Şevval Ensarioğlu - 22290742

GitHub Linki:

<https://github.com/SevvalEnsarioglu/blm4522-proje5-db-etl-cleaning>

İçindekiler

1 Giriş	3
1.1 Projenin Amacı	3
1.2 Kullanılan Araçlar ve Teknolojiler	3
2 Kullanılan Veritabanı: Northwind	4
2.1 Tablo Yapısı ve Kayıt Sayıları	4
2.2 ETL için Seçilen Tablolar	4
3 ETL Mimarisi ve Şema Tasarımı	5
3.1 Şemaların Oluşturulması	5
4 Extract — Veriyi Staging Alanına Çekme	6
4.1 Veri Aktarımı	6
4.2 Extract Sonrası Kayıt Doğrulama	6
5 Veri Kalitesi Analizi — Sorunların Tespiti	8
5.1 NULL Değer Analizi	8
5.1.1 Customers Tablosu	8
5.1.2 Orders Tablosu	8
5.2 Tespit Edilen Sorunlar	10
6 Transform — Veri Temizleme ve Dönüştürme	11
6.1 Boşluk Temizleme (TRIM)	11
6.2 Büyük/Küçük Harf Standardizasyonu	11
6.3 NULL Değerlerin Varsayılan Değerlerle Doldurulması	12
6.4 Negatif Stok Değerlerinin Düzeltilmesi	12
6.5 NULL Fiyatların Ortalama ile Doldurulması	12
6.6 Geçersiz Kargo Tarihlerinin Düzeltilmesi	13
6.7 Transform Özeti	13
7 Load — Temiz Veriyi Warehouse’a Yükleme	14
7.1 Warehouse Tablolarının Oluşturulması	14
7.2 Temiz Verinin Yükleneceği	14

8	Veri Kalitesi Raporları	15
8.1	Kaynak ve Hedef Kayıt Sayısı Karşılaştırması	15
8.2	NULL Düzeltme Raporu	16
8.3	Ülke Bazlı Müşteri Dağılımı Raporu	16
8.4	Kategori Bazlı Ürün ve Fiyat Raporu	17
8.5	Yıllık Sipariş ve Ciro Trendi	18
9	Sonuç ve Değerlendirme	19
9.1	Yapılan Çalışmaların Özeti	19
9.2	Öğrenilen Dersler	19
9.3	Kaynaklar	20

1 Giriş

Bu proje, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanmıştır. Proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde ETL (Extract, Transform, Load) süreçleri tasarlanmış ve uygulanmıştır.

Çalışmada gerçek dünya e-ticaret senaryolarını yansıtan Northwind örnek veritabanı kullanılmıştır. Bu veritabanı; müşteriler, siparişler, ürünler ve tedarikçilerden oluşan ilişkisel bir yapıya sahip olup toplamda 3.182 kayıt içermektedir.

ETL süreci üç temel aşamadan oluşmaktadır:

- **Extract (Çıkar):** Kaynak veritabanından veriyi staging alanına aktarma
- **Transform (Dönüştür):** Hatalı, eksik ve tutarsız verileri temizleme ve standartlaştırma
- **Load (Yükle):** Temizlenmiş veriyi hedef warehouse şemasına yükleme

1.1 Projenin Amacı

- Büyük veri kümelerindeki hatalı ve eksik kayıtları tespit etmek
- SQL kullanarak veri temizleme ve dönüştürme işlemleri uygulamak
- Staging ve Warehouse şema mimarisi tasarlamak
- ETL sürecinin her adımını raporlamak ve veri kalitesini ölçmek

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm	Kullanım Amacı
PostgreSQL	17	Ana veritabanı yönetim sistemi
Northwind DB	—	Örnek e-ticaret veritabanı (14 tablo, 3.182+ kayıt)
GitHub	—	Sürüm kontrolü ve proje takibi

Tablo 1: Kullanılan araçlar ve teknolojiler

2 Kullanılan Veritabanı: Northwind

Northwind veritabanı, bir e-ticaret şirketinin sipariş yönetim sistemini modelleyen klasik bir örnek veritabanıdır. GitHub üzerindeki PostgreSQL uyumlu versiyonu (pthom/northwind_psql) kullanılmıştır.

2.1 Tablo Yapısı ve Kayıt Sayıları

Veritabanındaki 14 tablo aşağıda listelenmiştir. ETL çalışması için 5 ana tablo seçilmiştir.

Tablo Adı	Kayıt Sayısı	Açıklama
order_details	2.155	Sipariş kalemleri — en büyük tablo
orders	830	Sipariş başlık bilgileri
customers	91	Müşteri kayıtları
products	77	Ürün bilgileri
suppliers	29	Tedarikçi kayıtları
categories	8	Ürün kategorileri
employees	9	Çalışan bilgileri
shippers	3	Kargo firmaları
territories	53	Bölge bilgileri
region	4	Bölge grupları
us_states	51	ABD eyalet bilgileri
customer_demographics	0	Müşteri demografik verileri
customer_customer_demo	0	Müşteri-demografi ilişkisi
employee_territories	49	Çalışan-bölge ilişkisi

Tablo 2: Northwind veritabanı tablo yapısı (14 tablo)

2.2 ETL için Seçilen Tablolar

ETL çalışmasında aşağıdaki 5 tablo kullanılmıştır:

Tablo	Kayıt	Kolon Sayısı	ETL Rolü
customers	91	11	Müşteri boyutu
orders	830	14	Sipariş gerçeği
order_details	2.155	5	Kalem gerçeği
products	77	10	Ürün boyutu
suppliers	29	12	Tedarikçi boyutu
TOPLAM	3.182	—	—

Tablo 3: ETL için seçilen 5 tablo

3 ETL Mimarisi ve Şema Tasarımı

Bu projede üç katmanlı bir şema mimarisi tasarlanmıştır:

Şema	Rol	Açıklama
public	Kaynak	Northwind ham verisi, değiştirilmez
staging	Ara katman	Ham verinin kopyalandığı, temizlendiği alan
warehouse	Hedef	Temizlenmiş, üretime hazır veri

Tablo 4: Üç katmanlı ETL şema mimarisi

3.1 Şemaların Oluşturulması

```
CREATE SCHEMA IF NOT EXISTS staging;  
CREATE SCHEMA IF NOT EXISTS warehouse;
```

Ardından staging tabloları, public şemasındaki yapılar kopyalanarak oluşturulmuştur:

```
CREATE TABLE staging.customers  
    AS SELECT * FROM public.customers WITH NO DATA;  
CREATE TABLE staging.orders  
    AS SELECT * FROM public.orders WITH NO DATA;  
CREATE TABLE staging.order_details  
    AS SELECT * FROM public.order_details WITH NO DATA;  
CREATE TABLE staging.products  
    AS SELECT * FROM public.products WITH NO DATA;  
CREATE TABLE staging.suppliers  
    AS SELECT * FROM public.suppliers WITH NO DATA;
```

4 Extract — Veriyi Staging Alanına Çekme

Extract aşamasında kaynak veritabanındaki (public şeması) tüm veriler staging şemasına aktarılmıştır. Bu sayede orijinal veri korunmuş, tüm temizleme işlemleri staging üzerinde yapılmıştır.

4.1 Veri Aktarımı

```
INSERT INTO staging.customers      SELECT * FROM public.customers;
INSERT INTO staging.orders         SELECT * FROM public.orders;
INSERT INTO staging.order_details SELECT * FROM public.
    order_details;
INSERT INTO staging.products       SELECT * FROM public.products;
INSERT INTO staging.suppliers      SELECT * FROM public.suppliers;
```

4.2 Extract Sonrası Kayıt Doğrulama

```
SELECT 'customers'      AS tablo, COUNT(*) AS kayit_sayisi
FROM staging.customers
UNION ALL
SELECT 'orders',          COUNT(*) FROM staging.orders
UNION ALL
SELECT 'order_details',   COUNT(*) FROM staging.
    order_details
UNION ALL
SELECT 'products',        COUNT(*) FROM staging.products
UNION ALL
SELECT 'suppliers',       COUNT(*) FROM staging.suppliers
;
```

Tablo	Kaynak (public)	Staging
customers	91	91
orders	830	830
order_details	2.155	2.155
products	77	77
suppliers	29	29
Toplam	3.182	3.182

Tablo 5: Extract aşaması sonrası kayıt sayısı doğrulaması

Sonuç: Tüm kayıtlar eksiksiz biçimde staging alanına aktarılmıştır.

5 Veri Kalitesi Analizi — Sorunların Tespiti

Transform aşamasına geçmeden önce staging verisinde hangi kalite sorunlarının mevcut olduğu tespit edilmiştir.

5.1 NULL Değer Analizi

5.1.1 Customers Tablosu

```
SELECT
    COUNT(*) AS toplam,
    COUNT(*) FILTER (WHERE company_name IS NULL) AS bos_sirket,
    COUNT(*) FILTER (WHERE contact_name IS NULL) AS bos_iletisim
    ,
    COUNT(*) FILTER (WHERE city IS NULL) AS bos_sehir,
    COUNT(*) FILTER (WHERE country IS NULL) AS bos_ulke,
    COUNT(*) FILTER (WHERE phone IS NULL) AS bos_telefon,
    COUNT(*) FILTER (WHERE postal_code IS NULL) AS
        bos_posta_kodu
FROM staging.customers;
```

5.1.2 Orders Tablosu

```
SELECT
    COUNT(*) AS toplam,
    COUNT(*) FILTER (WHERE customer_id IS NULL) AS
        bos_musteri,
    COUNT(*) FILTER (WHERE ship_city IS NULL) AS bos_sehir,
    COUNT(*) FILTER (WHERE ship_country IS NULL) AS bos_ulke,
    COUNT(*) FILTER (WHERE shipped_date IS NULL) AS
        bos_kargo_tarihi
FROM staging.orders;
\end{lstlisting}

\subsection{Veri Tutarsızlıklarının Kontrolü}
```

```

\begin{lstlisting}[style=sqlstyle]
-- Negatif/sifir fiyat kontrolu - products
SELECT product_id, product_name, unit_price
FROM staging.products
WHERE unit_price <= 0 OR unit_price IS NULL;

-- Negatif miktar kontrolu - order_details
SELECT * FROM staging.order_details
WHERE quantity <= 0 OR unit_price < 0;

-- Kargo tarihi siparis tarihinden once olanlar
SELECT order_id, order_date, shipped_date
FROM staging.orders
WHERE shipped_date < order_date;

-- Duplicate kontrolu - customers
SELECT company_name, COUNT(*)
FROM staging.customers
GROUP BY company_name
HAVING COUNT(*) > 1;

-- Buyuk/kucuk harf tutarsizligi
SELECT customer_id, company_name, country, city
FROM staging.customers
WHERE country != TRIM(country)
      OR city    != TRIM(city);

```

5.2 Tespit Edilen Sorunlar

Sorun Türü	Etkilenen Tablo	Açıklama
NULL şehir değerleri	customers, orders	Bazı kayıtlarda şehir bilgisi eksik
NULL ülke değerleri	customers	Ülke bilgisi eksik kayıtlar
NULL posta kodu	customers	Posta kodu eksik kayıtlar
Büyük/küçük harf	customers, suppliers	Ülke ve şehir isimlerinde tutarsızlık
Baştaki/sondaki boşluk	customers, suppliers, products	TRIM gerektiren alanlar
Negatif stok	products	Stok değeri sıfırın altında
Geçersiz kargo tarihi	orders	Kargo tarihi sipariş tarihinden önce

Tablo 6: Veri kalitesi analizi — tespit edilen sorunlar

6 Transform — Veri Temizleme ve Dönüştürme

6.1 Boşluk Temizleme (TRIM)

Tüm metin alanlarındaki baştaki ve sondaki boşluklar temizlenmiştir:

```
UPDATE staging.customers SET
    company_name = TRIM(company_name),
    contact_name = TRIM(contact_name),
    city         = TRIM(city),
    country      = TRIM(country),
    phone        = TRIM(phone);

UPDATE staging.suppliers SET
    company_name = TRIM(company_name),
    contact_name = TRIM(contact_name),
    city         = TRIM(city),
    country      = TRIM(country);

UPDATE staging.products SET
    product_name = TRIM(product_name);
```

6.2 Büyük/Küçük Harf Standardizasyonu

Ülke ve şehir adları INITCAP fonksiyonu ile ilk harf büyük olacak şekilde standartlaştırılmıştır:

```
UPDATE staging.customers SET
    country = INITCAP(country),
    city    = INITCAP(city);

UPDATE staging.suppliers SET
    country = INITCAP(country),
    city    = INITCAP(city);
```

Not: INITCAP dönüşümü sonucunda "USA" → "Usa", "UK" → "Uk" gibi bazı kı-

saltmalar etkilenmiştir. Bu durum, veri kalitesi raporunda tespit edilmiş olup ilerleyen aşamalarda özel kural tanımlama yoluyla giderilebilir.

6.3 NULL Değerlerin Varsayılan Değerlerle Doldurulması

```
-- NULL şehirleri 'Unknown' yap
UPDATE staging.customers
SET city = 'Unknown'
WHERE city IS NULL;

UPDATE staging.orders
SET ship_city = 'Unknown'
WHERE ship_city IS NULL;

-- NULL ülkeleri 'Unknown' yap
UPDATE staging.customers
SET country = 'Unknown'
WHERE country IS NULL;

-- NULL posta kodlarını 'N/A' yap
UPDATE staging.customers
SET postal_code = 'N/A'
WHERE postal_code IS NULL;
```

6.4 Negatif Stok Değerlerinin Düzeltilmesi

```
UPDATE staging.products
SET units_in_stock = 0
WHERE units_in_stock < 0;
```

6.5 NULL Fiyatların Ortalama ile Doldurulması

```
UPDATE staging.products
SET unit_price = (
```

```

SELECT AVG(unit_price)
FROM staging.products
WHERE unit_price IS NOT NULL
)
WHERE unit_price IS NULL;

```

6.6 Geçersiz Kargo Tarihlerinin Düzeltilmesi

Kargo tarihi sipariş tarihinden önce olan kayıtlar düzeltilmiştir:

```

UPDATE staging.orders
SET shipped_date = order_date
WHERE shipped_date < order_date;

```

6.7 Transform Özeti

İşlem	Etkilenen Tablo	Uygulanan Yöntem
Boşluk temizleme	customers, suppliers, products	TRIM()
Harf standardizasyonu	customers, suppliers	INITCAP()
NULL şehir doldurma	customers, orders	'Unknown' sabiti
NULL ülke doldurma	customers	'Unknown' sabiti
NULL posta kodu	customers	'N/A' sabiti
Negatif stok düzeltme	products	0 (sıfır) sabiti
NULL fiyat doldurma	products	AVG() ortalama
Geçersiz tarih düzeltme	orders	order_date ile eşitleme

Tablo 7: Transform aşamasında uygulanan veri temizleme işlemleri

7 Load — Temiz Veriyi Warehouse’a Yükleme

Transform aşaması tamamlandıktan sonra temizlenmiş veri, hedef warehouse şemasına yüklenmiştir.

7.1 Warehouse Tablolarının Oluşturulması

```
CREATE TABLE warehouse.customers
  AS SELECT * FROM staging.customers WITH NO DATA;
CREATE TABLE warehouse.orders
  AS SELECT * FROM staging.orders WITH NO DATA;
CREATE TABLE warehouse.order_details
  AS SELECT * FROM staging.order_details WITH NO DATA;
CREATE TABLE warehouse.products
  AS SELECT * FROM staging.products WITH NO DATA;
CREATE TABLE warehouse.suppliers
  AS SELECT * FROM staging.suppliers WITH NO DATA;
```

7.2 Temiz Verinin Yüklenmesi

```
INSERT INTO warehouse.customers      SELECT * FROM staging.
customers;
INSERT INTO warehouse.orders          SELECT * FROM staging.orders;
INSERT INTO warehouse.order_details  SELECT * FROM staging.
order_details;
INSERT INTO warehouse.products        SELECT * FROM staging.
products;
INSERT INTO warehouse.suppliers       SELECT * FROM staging.
suppliers;
```

Sonuç: Tüm temizlenmiş veriler warehouse şemasına eksiksiz biçimde yüklenmiştir. Kaynak, staging ve warehouse kayıt sayıları birbiriyle eşleşmektedir; bu durum veri bütünlüğünün korunduğunu doğrulamaktadır.

8 Veri Kalitesi Raporları

8.1 Kaynak ve Hedef Kayıt Sayısı Karşılaştırması

```
SELECT 'customers' AS tablo,
      (SELECT COUNT(*) FROM public.customers) AS kaynak,
      (SELECT COUNT(*) FROM warehouse.customers) AS temiz,
      (SELECT COUNT(*) FROM public.customers) -
      (SELECT COUNT(*) FROM warehouse.customers) AS fark
UNION ALL
SELECT 'orders',
      (SELECT COUNT(*) FROM public.orders),
      (SELECT COUNT(*) FROM warehouse.orders),
      (SELECT COUNT(*) FROM public.orders) -
      (SELECT COUNT(*) FROM warehouse.orders)
UNION ALL
SELECT 'order_details',
      (SELECT COUNT(*) FROM public.order_details),
      (SELECT COUNT(*) FROM warehouse.order_details),
      (SELECT COUNT(*) FROM public.order_details) -
      (SELECT COUNT(*) FROM warehouse.order_details)
UNION ALL
SELECT 'products',
      (SELECT COUNT(*) FROM public.products),
      (SELECT COUNT(*) FROM warehouse.products),
      (SELECT COUNT(*) FROM public.products) -
      (SELECT COUNT(*) FROM warehouse.products)
UNION ALL
SELECT 'suppliers',
      (SELECT COUNT(*) FROM public.suppliers),
      (SELECT COUNT(*) FROM warehouse.suppliers),
      (SELECT COUNT(*) FROM public.suppliers) -
      (SELECT COUNT(*) FROM warehouse.suppliers);
```


Tablo	Kaynak	Warehouse	Fark
customers	91	91	0
orders	830	830	0
order_details	2.155	2.155	0
products	77	77	0
suppliers	29	29	0
Toplam	3.182	3.182	0

Tablo 8: Kaynak ve warehouse kayıt sayısı karşılaştırması

Bulgu: Hiçbir kayıt kaybolmamıştır. ETL süreci veri bütünlüğünü tam olarak korumuştur.

8.2 NULL Düzeltme Raporu

```
SELECT 'NULL sehir duzeltilen (customers)' AS islem,
       COUNT(*) AS adet
FROM warehouse.customers WHERE city = 'Unknown'
UNION ALL
SELECT 'NULL ulke duzeltilen (customers)',
       COUNT(*) FROM warehouse.customers WHERE country = '
       Unknown'
UNION ALL
SELECT 'NULL posta kodu duzeltilen',
       COUNT(*) FROM warehouse.customers WHERE postal_code = 'N/
       A'
UNION ALL
SELECT 'Sifirlanan negatif stok (products)',
       COUNT(*) FROM warehouse.products WHERE units_in_stock =
       0;
```

8.3 Ülke Bazlı Müşteri Dağılımı Raporu

```
SELECT country, COUNT(*) AS musteri_sayisi
FROM warehouse.customers
GROUP BY country
ORDER BY musteri_sayisi DESC;
```

Ülke	Müşteri Sayısı
Usa	13
Germany	11
France	11
Brazil	9
Uk	7
Spain	5
Mexico	5
Venezuela	4
Italy	3
Diğerleri	23

Tablo 9: Temizlenmiş veri üzerinde ülke bazlı müşteri dağılımı

8.4 Kategori Bazlı Ürün ve Fiyat Raporu

```

SELECT
    c.category_name ,
    COUNT(p.product_id)                AS urun_sayisi ,
    ROUND(AVG(p.unit_price)::numeric, 2) AS ort_fiyat ,
    SUM(p.units_in_stock)              AS toplam_stok
FROM warehouse.products p
JOIN public.categories c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY urun_sayisi DESC;

```

Kategori	Ürün	Ort. Fiyat	Toplam Stok
Confections	13	\$25.16	386
Condiments	12	\$22.85	507
Beverages	12	\$37.98	559
Seafood	12	\$20.68	701
Dairy Products	10	\$28.73	393
Grains/Cereals	7	\$20.25	308
Meat/Poultry	6	\$54.01	165
Produce	5	\$32.37	100

Tablo 10: Kategori bazlı ürün sayısı, ortalama fiyat ve toplam stok raporu

8.5 Yıllık Sipariş ve Ciro Trendi

```
SELECT
    EXTRACT(YEAR FROM order_date) AS yil,
    COUNT(*) AS siparis_sayisi,
    ROUND(SUM(od.unit_price * od.quantity)::numeric, 2) AS
        toplam_ciro
FROM warehouse.orders o
JOIN warehouse.order_details od ON o.order_id = od.order_id
GROUP BY yil
ORDER BY yil;
```

Yıl	Sipariş Sayısı	Toplam Ciro (\$)
1996	405	226.298,50
1997	1.059	658.388,75
1998	691	469.771,34

Tablo 11: Yıllık sipariş sayısı ve ciro trendi (temizlenmiş veri)

Bulgu: 1997 yılı hem sipariş sayısı (1.059) hem de ciro (\$658.388) açısından en yüksek değerlere sahiptir. 1998 verisi yılın tamamını kapsamamaktadır.

9 Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL üzerinde Northwind e-ticaret veritabanı kullanılarak eksiksiz bir ETL süreci tasarlanmış ve uygulanmıştır.

9.1 Yapılan Çalışmaların Özeti

ETL Aşaması	Yapılan Çalışma	Sonuç
Şema Tasarımı	staging ve warehouse şemaları oluşturuldu	3 katmanlı mimari kuruldu
Extract	5 tablo public'ten staging'e aktarıldı	3.182 kayıt eksiksiz taşındı
Analiz	NULL, negatif, tutarsız veriler tespit edildi	7 farklı sorun kategorisi belirlendi
Transform	TRIM, INITCAP, NULL doldurma, tarih düzeltme	Tüm sorunlar giderildi
Load	Temiz veri warehouse'a yüklendi	3.182 kayıt bütünlüklü yüklendi
Raporlama	4 farklı veri kalitesi raporu oluşturuldu	Tablo, kategori, ülke ve yıl bazlı analizler

Tablo 12: ETL sürecinin özet tablosu

9.2 Öğrenilen Dersler

- Staging katmanı, kaynak veriyi koruyarak güvenli bir çalışma ortamı sağlar.
- INITCAP() fonksiyonu kısaltmalar (USA, UK) içeren verilerde dikkatli kullanılmalıdır; bu tür değerler için özel CASE kuralları tanımlanması daha doğru bir yaklaşımdır.
- ETL sürecinde kayıt sayısı doğrulaması, veri bütünlüğünün korunduğunu garantilemek için kritik bir adımdır.
- NULL değerler için seçilen varsayılan değerler ('Unknown', 'N/A', 0) iş kurallarına göre belirlenmelidir; her alan için evrensel bir çözüm yoktur.
- Veri temizleme, ham verinin analitik değerini doğrudan artırmaktadır — yıllık ciro ve kategori raporları ancak temizlenmiş veri üzerinde güvenilir sonuçlar vermektedir.

9.3 Kaynaklar

- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- Northwind PostgreSQL (GitHub): https://github.com/pthom/northwind_psql
- PostgreSQL Tutorial: <https://www.postgresqltutorial.com/>